CUDA Kernel 面试背题笔记

LeetCUDA Project

```
// ================================================
// notes-v2.cu — CUDA Kernel 面试背题笔记
// ================================================
//
// 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA), 涵盖:
// - 面试高频 CUDA kernel 的完整实现 (共 37 个 kernel)
// - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
// - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
// - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
//
// 10 个 Phase 覆盖:
// Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
// Phase 1 — 基础原语: Warp Reduce / Block Reduce (含 broadcast 增强版)
// Phase 2 — Elementwise: ReLU / Elementwise Add (基础 + float4 向量化)
// Phase 3 — Softmax: naive → safe(2-pass) → online(1-pass) + RMS/LayerNorm
// Phase 4 — GEMV: SGEMV K32/K128/K16 + HGEMV K32/K128/K16 (warp-per-row)
// Phase 5 — GEMM : SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMA m64n128k16
// Phase 6 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
// Phase 7 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
// Phase 8 — 杂项: Dot Product / Block All Reduce / Histogram
// Phase 9 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
//
// ================================================

// ================================================
// Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
// ================================================

// ---- GPU 架构速查 ----
//
// SM (Streaming Multiprocessor) 内部结构:
// - Warp Scheduler ×4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
// - Register File: 每 SM 65536 × 32-bit = 256KB
// - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
// - Tensor Cores: Hopper 每 SM 4 个, Blackwell 每 SM 8 个
// - Warp = 32 threads: 最小调度单元, SIMT 执行模型
//
// Memory Hierarchy 带宽数量级 (H100 参考):
// HBM3:         ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
// L2 Cache:     ~12 TB/s (50MB, 跨 SM 共享)
// L1/SMEM:      ~19 TB/s (每 SM ~228KB)
// Register:     ~0 延迟, ~100+ TB/s 等效带宽
//
// 关键瓶颈判断:
// Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
// Compute-bound: AI 足够大, 受限于计算吞吐
// Latency-bound: 线程不够多, 无法隐藏内存延迟
//
// Occupancy 公式:
// occupancy = active_warps / max_warps_per_SM
// 受限于: register/thread × threads/block + shared_memory/block

// ---- 常见优化手段速查清单 ----
//
// 1. Coalesced Memory Access (合并访问)
// - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
// - 否则产生多次事务 (最坏 32 次)
//
// 2. Tiling (分块)
// - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
// - GEMM: Block Tile (BM×BN) + K Tile (BK)
//
// 3. Vectorized Memory Access (向量化)
// - 使用 float4/half2 等向量类型, 减少 load/store 指令数
// - float4 = 128-bit, 单条指令加载 16 bytes
//
// 4. Thread Tile (寄存器分块)
// - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
// - 减少线程总数, 降低同步开销
//
// 5. Bank Conflict Avoidance
// - Shared memory 有 32 banks × 4 bytes
// - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
// - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
//
// 6. Pipeline / Double Buffering (流水线)
// - cp.async 异步拷贝下一批数据时同时做当前批的计算
// - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
//
// 7. Tensor Core (MMA / WGMMA)
// - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
// - WGMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
//
// 8. Warp Specialization (Hopper+)
// - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
// - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
//
// 9. TMA (Tensor Memory Accelerator, Hopper+)
// - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
// - 配合 cp.async.bulk 实现异步数据搬运

// ---- Roofline 分析公式 ----
//
// AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
//
// GEMM (M=N=K=4096):
// FLOPs = 2 × M × N × K = 2 × 4096³  137 GFLOPS
// Bytes  = (M×K + K×N + M×N) × sizeof(float)  200 MB
// AI     137G / 200M  685 FLOPS/Byte → compute-bound (远超 H100 的 ~50:1
// 墙)
//
// GEMV (M=4096, K=4096):
// FLOPs = 2 × M × K = 2 × 4096²  33 MFLOPS
// Bytes  = (M×K + K + M) × sizeof(float)  67 MB
// AI     33M / 67M  0.5 FLOPS/Byte → severely memory-bound
//
// Softmax (N=4096): AI  (5×N) / (2×N×4)  2.5 FLOPS/Byte → memory-bound

// ================================================
// Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
// ================================================
// 面试要点:
// - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared memory
// - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
// 注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
// - 为什么不用 __shfl_down_sync？xor 模式所有线程做相同工作量, 更均衡

#include <algorithm>
#include <cuda_fp16.h>
```

```cpp
#include <cuda_runtime.h>
#include <float.h>
#include <stdio.h>
#include <stdlib.h>

#define WARP_SIZE 32
#define INT4(value) (reinterpret_cast<int4 *>(&(value))[0])
#define FLOAT4(value) (reinterpret_cast<float4 *>(&(value))[0])
#define HALF2(value) (reinterpret_cast<half2 *>(&(value))[0])

// ==================================================
// Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
// ==================================================

// Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
// 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
// 复杂度 O(logN), N=32 时仅需 5 步
// 双模板参数: T=数据类型, kWarpSize=segment width (默认 32)
// 第四个参数 kWarpSize 是 segment width: 限制 shuffle 在同一 segment 内
// 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
template <typename T = float, const int kWarpSize = WARP_SIZE>
__device__ __forceinline__ T warp_reduce_sum(T val) {
#pragma unroll
  for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
    val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
  }
  return val;
}

// Warp Reduce Max — generic
template <typename T = float, const int kWarpSize = WARP_SIZE>
__device__ __forceinline__ T warp_reduce_max(T val) {
#pragma unroll
  for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
    val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
  }
  return val;
}

// Warp Reduce Sum — FP16 (保留特化版本, 用于 HGEMV)
template <const int kWarpSize = WARP_SIZE>
__device__ __forceinline__ half warp_reduce_sum_f16(half val) {
#pragma unroll
  for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
    val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
  }
  return val;
}

// ==================================================
// Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp0)
// ==================================================

// Block Reduce Sum — FP32 (增强版, 带 broadcast)
// 两级归约流程:
// 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
// 2. warp leader (lane=0) 写入 shared memory
// 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
// 4. warp0 内再做一次 warp_reduce → 得到最终结果
// 5. __shfl_sync broadcast 到所有线程 (关键! 否则只有 warp0 知道结果)
template <const int NUM_THREADS = 256>
__device__ float block_reduce_sum_f32(float val) {
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  int warp = threadIdx.x / WARP_SIZE;
  int lane = threadIdx.x % WARP_SIZE;
  static __shared__ float shared[NUM_WARPS];

  float value = warp_reduce_sum<WARP_SIZE>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
  value = warp_reduce_sum<NUM_WARPS>(value);
  // 关键: broadcast 结果到所有线程, 后续用 result
  // 做除法等操作时所有线程都能拿到
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// Block Reduce Max — FP32 (增强版, 带 broadcast)
template <const int NUM_THREADS = 256>
__device__ float block_reduce_max_f32(float val) {
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  int warp = threadIdx.x / WARP_SIZE;
  int lane = threadIdx.x % WARP_SIZE;
  static __shared__ float shared[NUM_WARPS];

  float value = warp_reduce_max<WARP_SIZE>(val);
  if (lane == 0)
    shared[warp] = value;
  __syncthreads();
  value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
  value = warp_reduce_max<NUM_WARPS>(value);
  value = __shfl_sync(0xffffffff, value, 0, 32);
  return value;
}

// Block Reduce Sum — FP32 (notes-v1 原版, 保留用于兼容旧 kernel)
// 注意: 此版本没有 broadcast, 仅 warp0 持有最终结果
template <const int NUM_THREADS = 128>
__device__ __forceinline__ float block_reduce_sum(float val) {
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  int warp = threadIdx.x / WARP_SIZE;
  int lane = threadIdx.x % WARP_SIZE;
  static __shared__ float shared[NUM_WARPS];

  val = warp_reduce_sum<WARP_SIZE>(val);
  if (lane == 0)
    shared[warp] = val;
  __syncthreads();
  val = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
  val = warp_reduce_sum<NUM_WARPS>(val);
  return val;
}

template <const int NUM_THREADS = 128>
__device__ __forceinline__ float block_reduce_max(float val) {
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  int warp = threadIdx.x / WARP_SIZE;
  int lane = threadIdx.x % WARP_SIZE;
  static __shared__ float shared[NUM_WARPS];

  val = warp_reduce_max<WARP_SIZE>(val);
  if (lane == 0)
    shared[warp] = val;
  __syncthreads();
  val = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
  val = warp_reduce_max<NUM_WARPS>(val);
  return val;
}

// ==================================================
// Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
// ==================================================
// 面试要点:
// - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
// - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
// - grid/block 维度设计: grid(N/threads), block(threads), 一维即可

// ---- ReLU: y = max(0, x) ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu(float *x, float *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    y[idx] = fmaxf(0.0f, x[idx]);
```

```
}

// ReLU + float4 向量化：每个线程处理 4 个元素，减少 75% 的 load/store 指令
// Block: (256, 1, 1)
// block(64)×4(float4)=256 元素/block，与基础版吞吐相同
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1)
// 注意：该版本默认地址满足 float4 对齐；最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/relu/relu.cu
__global__ void relu_vec4(float *x, float *y, int N) {
  int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
  if (idx < N) {
    float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
    float4 reg_y;
    reg_y.x = fmaxf(0.0f, reg_x.x);
    reg_y.y = fmaxf(0.0f, reg_x.y);
    reg_y.z = fmaxf(0.0f, reg_x.z);
    reg_y.w = fmaxf(0.0f, reg_x.w);
    FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
  }
}

// ---- Elementwise Add: c = a + b ----
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add(float *a, float *b, float *c, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    c[idx] = a[idx] + b[idx];
}

// Elementwise Add + float4 向量化
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
// 注意：主路径要求 4 元素对齐；尾部不足 4 个元素时回退到标量处理
// source: LeetCUDA/kernels/elementwise/elementwise.cu
__global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
  int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
  if ((idx + 3) < N) {
    float4 reg_a = FLOAT4(a[idx]);
    float4 reg_b = FLOAT4(b[idx]);
    float4 reg_c;
    reg_c.x = reg_a.x + reg_b.x;
    reg_c.y = reg_a.y + reg_b.y;
    reg_c.z = reg_a.z + reg_b.z;
    reg_c.w = reg_a.w + reg_b.w;
    FLOAT4(c[idx]) = reg_c;
  } else if (idx < N) {
    for (int i = 0; (idx + i) < N; i++) {
      c[idx + i] = a[idx + i] + b[idx + i];
    }
  }
}

// ================================================
// Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
// ================================================
// 面试要点：
// - Softmax 三种实现递进：naive（溢出）→ safe（2-pass，max 减法）→
// online（1-pass，增量更新）
// - Online Softmax 是 FlashAttention 的数学基础
// - RMS Norm vs Layer Norm：RMS 只需 1 次 reduce，Layer Norm 需要 2 次
// （mean
// + variance）
// - Per-token 设计：一个 block 处理一个 token，无需跨 block 同步

// ---- Online Softmax 辅助结构 ----
// MD struct：存储 running max (m) 和 running denominator (d)
// 算法来源："Online normalizer calculation for softmax" (arXiv
// :1805.02867)
// 核心递推公式：
// m_new = max(m_old, m_cur)
// d_new = d_old * exp(m_old - m_new) + d_cur * exp(m_cur - m_new)
// FlashAttention 中用同样的公式做 online rescaling：
// O_new = diag(exp(m_old - m_new)) * O_old + exp(m_cur - m_new) * P@V
struct __align__(8) MD {
  float m; // running max
  float d; // running denominator (sum of exp(x - max))
};

// Warp Reduce for Online Softmax
// 与普通 reduce 不同：归约时需同时更新 m 和 d（因为 max 在不断变大）
template <const int kWarpSize = WARP_SIZE>
__device__ __forceinline__ MD warp_reduce_md_op(MD value) {
  unsigned int mask = 0xffffffff;
#pragma unroll
  for (int stride = kWarpSize >> 1; stride >= 1; stride >>= 1) {
    MD other;
    other.m = __shfl_xor_sync(mask, value.m, stride);
    other.d = __shfl_xor_sync(mask, value.d, stride);

    bool value_bigger = (value.m > other.m);
    MD bigger_m = value_bigger ? value : other;
    MD smaller_m = value_bigger ? other : value;

    // 关键：更新 d 时需要 rescale 旧的 d 到新的 max 尺度下
    value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m
      );
    value.m = bigger_m.m;
  }
  return value;
}

// ================================================
// Phase 3a: Softmax — 三级递进（面试核心考点）
// ================================================
// 面试常问：「Softmax 有哪些实现方式？各有什么优缺点？」
// 回答线索：naive(溢出) → safe(2-pass, 稳定) → online(1-pass, 为 FA 打基
// 础)

// ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定)----
// grid(S*h/h, h), block(h), 一个 block 处理一个 token
// 问题：x 值很大时 exp(x) 溢出为 inf
template <const int NUM_THREADS = 256>
// Grid:  (S, 1, 1), S=batch*seq_len,
// DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个
// block
// 处理一个 token
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void softmax_f32_per_token_kernel(float *x, float *y, int N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
  float exp_sum = block_reduce_sum_f32<NUM_THREADS>(exp_val);
  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定)----
// 面试重点：为什么 Softmax 需要 Safe？
// - exp(89)  4.5e38 已接近 float32 上限，exp(100) 直接 inf
// - 减去 max 后：exp(x - max)  exp(0) = 1.0，永不超过 1
// - 数学等价性：softmax(x) = softmax(x - c) 对任意常数 c 成立
// - 代价：2 次 block reduce（先 max，再 sum），但仍 O(N/B) 高效
template <const int NUM_THREADS = 256>
// Grid:  (S, 1, 1)
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void safe_softmax_f32_per_token_kernel(float *x, float *y, int
    N) {
  const int tid = threadIdx.x;
  const int idx = blockIdx.x * blockDim.x + tid;

  // Pass 1: block reduce max — 找最大值
  float val = (idx < N) ? x[idx] : -FLT_MAX;
  float max_val = block_reduce_max_f32<NUM_THREADS>(val);
```

```cpp

  // Pass 2: exp(x - max) → block reduce sum
  float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
  float exp_sum = block_reduce_sum_f32<NUM_THREADS>(exp_val);

  if (idx < N)
    y[idx] = exp_val / exp_sum;
}

// ---- Level 3: Online Safe Softmax (1-pass, FlashAttention 的数学基础) ----
// 面试重点: Online Softmax 为什么重要?
// - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
// - Online Softmax 用递推公式合并为 1 遍遍历
// - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
// - 正是 FlashAttention 中 "online rescaling":
// O_new = diag(exp(m_old - m_new)) * O_old + exp(m_cur - m_new) * P@V
// 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
// 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
template <const int NUM_THREADS = 256>
// Grid:  (S, 1, 1)
// Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
// source: LeetCUDA/kernels/softmax/softmax.cu
__global__ void online_safe_softmax_f32_per_token_kernel(const float *x,
                                                         float *y, int N) {
  int local_tid = threadIdx.x;
  int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
  const int WARP_NUM = NUM_THREADS / WARP_SIZE;
  int warp_id = local_tid / WARP_SIZE;
  int lane_id = local_tid % WARP_SIZE;

  // 初始化: 每个线程持有一个 (max, denom) 对
  MD val;
  val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
  val.d = global_tid < N ? 1.0f : 0.0f;

  // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
  __shared__ MD shared[WARP_NUM];
  MD res = warp_reduce_md_op<WARP_SIZE>(val);

  if (lane_id == 0)
    shared[warp_id] = res;
  __syncthreads();

  // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
  // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个
  // block 可见
  if (local_tid < WARP_SIZE) {
    MD block_res =
        local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
    block_res = warp_reduce_md_op<WARP_NUM>(block_res);
    if (local_tid == 0) {
      shared[0] = block_res;
    }
  }
  __syncthreads();

  // 用全局 max 和 denom 做最终 softmax
  MD final_res = shared[0];
  float d_total_inverse = __fdividef(1.0f, final_res.d);
  // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
  if (global_tid < N) {
    y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
  }
}

// ================================================
// Phase 3b: RMS Normalization (1-pass reduce)
// ================================================
// 面试要点:
// - RMS Norm: y = x / rms(x) * g, rms(x) = sqrt(mean(x²))
// - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
// - Llama 系列使用 RMS Norm
// - grid(N, K/K), block(K): 一行一个 block

template <const int NUM_THREADS = 128>
// Grid:  (N, 1, 1), N=batch*seq_len, 每行一个 block
// Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
__global__ void rms_norm(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = bid * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;
  float variance = value * value;
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();
  if (idx < N * K)
    y[idx] = (value * s_variance) * g;
}

// RMS Norm + float4
template <const int NUM_THREADS = 128 / 4>
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/rms-norm/rms_norm.cu
__global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = (bid * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
                                    reg_x.z * reg_x.z + reg_x.w * reg_x.w)
                                 : 0.0f;
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();
  float4 reg_y;
  reg_y.x = reg_x.x * s_variance * g;
  reg_y.y = reg_x.y * s_variance * g;
  reg_y.z = reg_x.z * s_variance * g;
  reg_y.w = reg_x.w * s_variance * g;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ================================================
// Phase 3c: Layer Normalization (2-pass reduce)
// ================================================
// 面试要点:
// - Layer Norm: y = (x - mean) / std * g + b
// - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)²)/K)
// - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance

template <const int NUM_THREADS = 128>
// Grid:  (N, 1, 1), 一行一个 block
// Block: (128, 1, 1), NUM_THREADS=K=128
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
```

```cpp
__global__ void layer_norm(float *x, float *y, float g, float b, int N,
    int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = bid * blockDim.x + threadIdx.x;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float value = (idx < N * K) ? x[idx] : 0.0f;

  // Pass 1: compute mean
  float sum = block_reduce_sum<NUM_THREADS>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads(); // 必须等待 s_mean 对所有线程可见

  // Pass 2: compute variance = (x - mean)²
  float variance = (value - s_mean) * (value - s_mean);
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads(); // 必须等待 s_variance 对所有线程可见

  if (idx < N * K)
    y[idx] = ((value - s_mean) * s_variance) * g + b;
}

// Layer Norm + float4
template <const int NUM_THREADS = 128 / 4>
// Grid:  (N, 1, 1)
// Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
// 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
// source: LeetCUDA/kernels/layer-norm/layer_norm.cu
__global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
                                int K) {
  int tid = threadIdx.x;
  int bid = blockIdx.x;
  int idx = (bid * blockDim.x + threadIdx.x) * 4;
  const float epsilon = 1e-5f;

  __shared__ float s_mean;
  __shared__ float s_variance;
  float4 reg_x = FLOAT4(x[idx]);
  float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) :
      0.0f;

  float sum = block_reduce_sum<NUM_THREADS>(value);
  if (tid == 0)
    s_mean = sum / (float)K;
  __syncthreads();

  float4 reg_x_hat;
  reg_x_hat.x = reg_x.x - s_mean;
  reg_x_hat.y = reg_x.y - s_mean;
  reg_x_hat.z = reg_x.z - s_mean;
  reg_x_hat.w = reg_x.w - s_mean;
  float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y
      +
                   reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
  variance = block_reduce_sum<NUM_THREADS>(variance);
  if (tid == 0)
    s_variance = rsqrtf(variance / (float)K + epsilon);
  __syncthreads();

  float4 reg_y;
  reg_y.x = reg_x_hat.x * s_variance * g + b;
  reg_y.y = reg_x_hat.y * s_variance * g + b;
  reg_y.z = reg_x_hat.z * s_variance * g + b;
  reg_y.w = reg_x_hat.w * s_variance * g + b;
  if (idx < N * K)
    FLOAT4(y[idx]) = reg_y;
}

// ================================================
// Phase 4: GEMV — 矩阵向量乘 (纯 memory-bound 算子, warp-per-row 策略)
// ================================================
// 面试要点:
// - GEMV 是典型的 memory-bound 算子: AI  O(1), 瓶颈在内存带宽
// - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
// - 不同 K 值对应不同分块策略:
// K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
// K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
// K=16 < 32: 一个 warp 不够, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
//
// a: M×K, x: K×1, y: M×1, 计算: y = a * x

// ---- SGEMV K32: 基础 warp-per-row ----
// 设计: block(32, 4), blockDim.x=WARP_SIZE=K, blockDim.y=4 行/block
// grid(M/4), 每个 warp 负责一行
// K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
// Grid:  ((M + 3) / 4, 1, 1), 每 block 处理 4 行
// Block: (32, 4, 1), 每行 1 warp
// 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;    // 0~31
  int m = bx * blockDim.y + ty; // 全局行号
  if (m < M) {
    float sum = 0.0f;
    int NUM_WARPS = (K + WARP_SIZE - 1) / WARP_SIZE;
#pragma unroll
    for (int w = 0; w < NUM_WARPS; ++w) {
      int k = w * WARP_SIZE + lane;
      sum += a[m * K + k] * x[k];
    }
    sum = warp_reduce_sum<WARP_SIZE>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K128: float4 向量化 ----
// 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
  int tx = threadIdx.x; // 0~31
  int ty = threadIdx.y; // 0~3
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE; // 0~31
  int m = blockDim.y * bx + ty;

  if (m < M) {
    float sum = 0.0f;
    // K/128 个 warp 覆盖 K 维
    int NUM_WARPS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
#pragma unroll
    for (int w = 0; w < NUM_WARPS; ++w) {
      int k = (w * WARP_SIZE + lane) * 4;
      float4 reg_x = FLOAT4(x[k]);
      float4 reg_a = FLOAT4(a[m * K + k]);
      sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
              reg_a.w * reg_x.w);
    }
    sum = warp_reduce_sum<WARP_SIZE>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
// 面试亮点: K=16 < 32, 一个 warp 可以处理多行
```

```cpp
// ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理
// row1
template <const int ROW_PER_WARP = 2>
// Grid:  ((M + 7) / 8, 1, 1), NUM_ROWS=8
// Block: (32, 4, 1)
// 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
// source: LeetCUDA/kernels/sgemv/sgemv.cu
__global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
  constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP;
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;
  int k = lane % K_WARP_SIZE; // 0~15
  int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
  if (m < M) {
    float sum = A[m * K + k] * x[k];
    sum = warp_reduce_sum<K_WARP_SIZE>(sum);
    // 注意: 判断条件是 k == 0, 不是 lane == 0!
    if (k == 0)
      y[m] = sum;
  }
}

// ---- HGEMV K32: FP16 版本 ----
// 策略与 SGEMV K32 完全一致, 仅数据类型从 float 变为 half
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
// source: LeetCUDA/kernels/hgemv/hgemv.cu
__global__ void hgemv_k32_f16_kernel(half *a, half *x, half *y, int M,
                                     int K) {
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;
  int m = bx * blockDim.y + ty;
  if (m < M) {
    half sum = 0.0f;
    int NUM_WARPS = (K + WARP_SIZE - 1) / WARP_SIZE;
#pragma unroll
    for (int w = 0; w < NUM_WARPS; ++w) {
      int k = w * WARP_SIZE + lane;
      sum += a[m * K + k] * x[k];
    }
    sum = warp_reduce_sum_f16<WARP_SIZE>(sum); // FP16 warp reduce
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- HGEMV K128: FP16 + half2 向量化 ----
// 每个线程处理 4 个 half (2 个 half2), 一个 warp 覆盖 128 个元素
// Grid:  ((M + 3) / 4, 1, 1)
// Block: (32, 4, 1)
// 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 half2 打包访问前提
// source: LeetCUDA/kernels/hgemv/hgemv.cu
__global__ void hgemv_k128_f16x4_kernel(half *a, half *x, half *y, int M,
                                        int K) {
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;
  int m = blockDim.y * bx + ty;

  if (m < M) {
    half sum = 0.0f;
    int NUM_WARPS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
#pragma unroll
    for (int w = 0; w < NUM_WARPS; ++w) {
      int k = (w * WARP_SIZE + lane) * 4;
      half2 reg_x_0 = HALF2(x[k + 0]);
      half2 reg_x_1 = HALF2(x[k + 2]);
      half2 reg_a_0 = HALF2(a[m * K + k + 0]);
      half2 reg_a_1 = HALF2(a[m * K + k + 2]);
      sum += (reg_x_0.x * reg_a_0.x + reg_x_0.y * reg_a_0.y +
              reg_x_1.x * reg_a_1.x + reg_x_1.y * reg_a_1.y);
    }
    sum = warp_reduce_sum_f16<WARP_SIZE>(sum);
    if (lane == 0)
      y[m] = sum;
  }
}

// ---- HGEMV K16: FP16 + ROW_PER_WARP=2 ----
template <const int ROW_PER_WARP = 2>
// Grid:  ((M + 7) / 8, 1, 1)
// Block: (32, 4, 1)
// 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
// source: LeetCUDA/kernels/hgemv/hgemv.cu
__global__ void hgemv_k16_f16_kernel(half *A, half *x, half *y, int M,
                                     int K) {
  constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP;
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int bx = blockIdx.x;
  int lane = tx % WARP_SIZE;
  int k = lane % K_WARP_SIZE;
  int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
  if (m < M) {
    half sum = A[m * K + k] * x[k];
    sum = warp_reduce_sum_f16<K_WARP_SIZE>(sum);
    if (k == 0)
      y[m] = sum;
  }
}

// ================================================
// Phase 5: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
// ================================================
// 面试要点 (GEMM 优化五层金字塔):
// Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
// Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
// 个元素, 提高计算密度
// Level 3 — Vectorize (向量化访存): float4/half2, 减少
// load/store 指令数
// Level 4 — Tensor Core (MMA
// m16n8k16): 硬件矩阵乘单元, warp 级指令
// Level 5 — Warp Specialization +
// TMA (WGMMA m64n128k16): Hopper 异步执行
//
// 计算密度递进:
// Level 1: AI  B_K / (2×sizeof)  32/8 = 4 → 仍是 memory-bound
// Level 2: AI  TM×TN×B_K / (2×sizeof)  8×8×8/8 = 64 → compute-bound
// Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐

// ================================================
// Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
// ================================================

// ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
// 最基础的 tiling 实现, 演示 shared memory 的核心用法
// BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
// Grid:  ((N + 31) / 32, (M + 31) / 32, 1)
// Block: (32, 32, 1)
// source: LeetCUDA/kernels/sgemm/sgemm.cu
__global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
  constexpr int BM = 32;
  constexpr int BN = 32;
  constexpr int BK = 32;
  __shared__ float s_a[BM][BK], s_b[BK][BN];

  int bx = blockIdx.x;
  int by = blockIdx.y;
```

```
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int tid = threadIdx.y * blockDim.x + tx;

  // 线程到 smem 的映射: 32×32 线程, 每个线程加载 a 和 b 各 1 个元素
  int load_smem_a_m = tid / 32;
  int load_smem_a_k = tid % 32;
  int load_smem_b_k = tid / 32;
  int load_smem_b_n = tid % 32;
  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  float sum = 0.f;
  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
    int load_gmem_b_k = bk * BK + load_smem_b_k;
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
    __syncthreads(); // 确保整个 smem tile 加载完毕

#pragma unroll
    for (int k = 0; k < BK; ++k) {
      int comp_smem_a_m = load_smem_a_m;
      int comp_smem_b_n = load_smem_b_n;
      sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
    }
    __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
  }
  int store_gmem_c_m = load_gmem_a_m;
  int store_gmem_c_n = load_gmem_b_n;
  int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
  c[store_gmem_c_addr] = sum;
}

// ---- Level 2+3: SGEMM — Block Tile 128×128 + Thread Tile 8×8 + float4 ----
// 三层 tile hierarchy: Block Tile → Thread Tile → K Tile
// BM=BN=128, TM=TN=8, BK=8, block(16, 16)=256 threads
// 每个线程计算 TM×TN=64 个元素, 大幅提升计算密度
// Grid:  ((N + 127) / 128, (M + 127) / 128, 1)
// Block: (16, 16, 1), 256 线程
// source: LeetCUDA/kernels/sgemm/sgemm.cu
__global__ void sgemm_thread_tile_vec4(float *a, float *b, float *c, int M,
                                       int N, int K) {
  constexpr int BM = 128;
  constexpr int BN = 128;
  constexpr int BK = 8;
  constexpr int TM = 8;
  constexpr int TN = 8;

  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int tid = threadIdx.y * blockDim.x + tx;

  // smem: 2×128×8×4 = 8KB (只有 1/4 的 L1/SMEM!)
  __shared__ float s_a[BM][BK], s_b[BK][BN];

  // 线程到 smem 的映射 — 256 线程协作加载 128×8 的矩阵
  // s_a[BM][BK]=[128][8]: 每行 8 元素, 2 线程/行(float4)→ 256 线程覆盖 128 行
  int load_smem_a_m = tid / 2;
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 4;
  // s_b[BK][BN]=[8][128]: 每行 128 元素, 32 线程/行(float4)→ 256 线程覆盖 8 行
  int load_smem_b_k = tid / 32;
  int load_smem_b_n = (tid % 32) * 4;

  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  // 寄存器分块: 每个线程累积 TM×TN=64 个部分和
  float r_c[TM][TN] = {0.0};

  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);

    int load_gmem_b_k = bk * BK + load_smem_b_k;
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
    __syncthreads();

#pragma unroll
    for (int k = 0; k < BK; k++) {
#pragma unroll
      for (int m = 0; m < TM; m++) {
#pragma unroll
        for (int n = 0; n < TN; n++) {
          int comp_smem_a_m = ty * TM + m;
          int comp_smem_b_n = tx * TN + n;
          r_c[m][n] += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
        }
      }
    }
    __syncthreads();
  }

  // 写回: float4 向量化 store
#pragma unroll
  for (int m = 0; m < TM; ++m) {
    int store_gmem_c_m = by * BM + ty * TM + m;
#pragma unroll
    for (int n = 0; n < TN; n += 4) {
      int store_gmem_c_n = bx * BN + tx * TN + n;
      int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
      FLOAT4(c[store_gmem_c_addr]) = FLOAT4(r_c[m][n]);
    }
  }
}

// ---- Level 2+3 (FP16): HGEMM — Thread Tile 8×8 + half2 向量化 ----
// FP16 版本: BM=BN=128, TM=TN=8, BK=8, block(16, 16)
// half2 向量化: 每次加载 2 个 half (32-bit) 到寄存器
// 面试注意: 这里的优化重点是 half2 访存向量化; 算术语义以当前 half 运算符实现为准
template <const int BM = 128, const int BN = 128, const int BK = 8,
          const int TM = 8, const int TN = 8>
// Grid:  ((N + 127) / 128, (M + 127) / 128, 1)
// Block: (16, 16, 1), 256 线程
// source: LeetCUDA/kernels/hgemm/naive/hgemm.cu
__global__ void hgemm_t_8x8_sliced_k_f16x4_kernel(half *a, half *b, half *c,
                                                  int M, int N, int K) {
  int bx = blockIdx.x;
  int by = blockIdx.y;
  int tx = threadIdx.x;
  int ty = threadIdx.y;
  int tid = threadIdx.y * blockDim.x + tx;

  // FP16 smem: 2×128×8×2 = 4KB (是 FP32 的一半!)
  __shared__ half s_a[BM][BK], s_b[BK][BN];

  // 线程到 smem 映射与 FP32 版本相同
  int load_smem_a_m = tid / 2;
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 4;
  int load_smem_b_k = tid / 32;
  int load_smem_b_n = (tid % 32) * 4;

  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n = bx * BN + load_smem_b_n;

  // 注意: 寄存器分块用 half 类型
```

```
  half r_c[TM][TN] = {__float2half(0.0f)};

  for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
    int load_gmem_a_k = bk * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
    // half2 向量化: 一次加载 2 个 half = 4 bytes
    HALF2(s_a[load_smem_a_m][load_smem_a_k]) = HALF2(a[load_gmem_a_addr
      ]);

    int load_gmem_b_k = bk * BK + load_smem_b_k;
    int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
    HALF2(s_b[load_smem_b_k][load_smem_b_n]) = HALF2(b[load_gmem_b_addr
      ]);
    __syncthreads();

#pragma unroll
    for (int k = 0; k < BK; k++) {
#pragma unroll
      for (int m = 0; m < TM; m++) {
#pragma unroll
        for (int n = 0; n < TN; n++) {
          int comp_smem_a_m = ty * TM + m;
          int comp_smem_b_n = tx * TN + n;
          r_c[m][n] += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
        }
      }
    }
    __syncthreads();
  }

#pragma unroll
  for (int m = 0; m < TM; ++m) {
    int store_gmem_c_m = by * BM + ty * TM + m;
#pragma unroll
    for (int n = 0; n < TN; n += 2) {
      int store_gmem_c_n = bx * BN + tx * TN + n;
      int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
      HALF2(c[store_gmem_c_addr]) = HALF2(r_c[m][n]);
    }
  }
}

// ================================================
// Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
// ================================================
// 面试要点:
// - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
// - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
// - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
// 寄存器)
// - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数
    据
// - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
//
//   TN 布局详解 (面试高频考点):
// TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
// BLAS 源自 Fortran, 默认列优先 (column-major) 存储
// N = Normal (列优先, BLAS 原生格式)
// T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
// 第一个字母 → A 的 op, 第二个字母 → B 的 op
// 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
// BLAS=Normal)  即: C = op(A) × op(B) = A^T × B? 不对!  在 row-major
// 视角下: TN = A row-major [M×K], B col-major [N×K] 在 cuBLAS
// 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
// T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
// N on B: B 已经是 BLAS 原生的 col-major, 无需转置
//
// 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
// T = row-major (行优先, 对 BLAS 来说是 transposed)
// N = col-major (列优先, BLAS native = normal)
//
// NN 布局对比: C[M×N] = A[M×K] × B[K×N]
// - A: row-major [M, K], B: row-major [K, N]
// - 两者都是行优先 → BLAS 视角都是 T → cuBLAS: (T, T)
// - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
//
// TN 布局: C[M×N] = A[M×K] × B[N×K] (A 行优先, B 列优先)
// - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
// - B [N×K]: col-major → 全局索引 B[n*K + k] (  内维是 K!)
// 物理含义: 存的是原 B[K×N] 的转置 B^T
// - 优势: B 已是 col-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
// MMA 指令: mma.sync.aligned.m16n8k16.row.col
// - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
//
// WGMMA (Warp Group MMA, Hopper+):
// - warpgroup 级指令 (128 threads = 4 warps), 异步执行
// - m64n128k16: 一次处理 64×128×16 的矩阵乘 (比 MMA 大 32 倍)
// - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
// threads) 做计算
// - TMA: 硬件 DMA, 零寄存器开销, 支持 2D/3D 寻址

// ================================================
// Phase 5b-1: MMA PTX 宏定义
// ================================================

// ---- gmem → smem: cp.async ----
#define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n"
    ::)
#define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
#define CP_ASYNC_WAIT_GROUP(n)
  asm volatile("cp.async.wait_group %0;\n" ::"n"(n))

// cp.async.cg: bypass L1, 写入 L2 (适合 GEMM 中只读一次的数据)
// cp.async.ca: cache all, L1+L2 (适合需要多次复用的数据)
// 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
#define CP_ASYNC_CG(dst, src, bytes)
  asm volatile(
      "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" ::"r"(dst),
      "l"(src), "n"(bytes))

// ---- ldmatrix: smem → register (Tensor Core 专用)----
// ldmatrix.sync.aligned.xN.m8n8.shared.b16
// 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
// aligned: 要求 128-bit 对齐
// trans:  转置加载 (用于 col-major 的 B 矩阵)
#define LDMATRIX_X4(R0, R1, R2, R3, addr)
  asm volatile(
      "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n
      "
      : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3)
      : "r"(addr))

#define LDMATRIX_X2(R0, R1, addr)
  asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\
    n"
               : "=r"(R0), "=r"(R1)
               : "r"(addr))

// ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
// FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用
    trans
// 加载
#define LDMATRIX_X2_T(R0, R1, addr)
  asm volatile(
      "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"
      : "=r"(R0), "=r"(R1)
      : "r"(addr))

// ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
// m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
// row.col: A 是 row-major, B 是 col-major
// f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.
    f32)
// 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
// C 矩阵大小 16×8=128 元素 = 64 个 half = 2 个 uint32
#define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)
  asm volatile(
      "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2,
      %3, "
      "%4, %5}, {%6, %7}, {%8, %9};\n"
```

```
      : "=r"(RD0), "=r"(RD1)
      : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0),
        "r"(RC1))

// ---- MMA 辅助函数 ----
// div_ceil: 整数除法向上取整
#define HOST_DEVICE_INLINE __device__ __host__ inline
HOST_DEVICE_INLINE int div_ceil(int a, int b) {
  return (a % b != 0) ? (a / b + 1) : (a / b);
}

// ================================================
// Phase 5b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局
// ================================================
// 面试重点 — Tile Hierarchy:
// MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
// MMA Tile (warp):   2×4=8 个 MMA atom = [32×32] (一个 warp 的计算)
// Warp Tile:         4×4=16 warps = [128×128] (整个 block 的计算)
// 线程映射:          2×4=8 warps, 每个 warp 4×1=4 个 warp tile
// 实际: MMA_TILE_M=2, MMA_TILE_N=4, WARP_TILE_M=4, WARP_TILE_N=4
// → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
//
// TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
// - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
// - B[N][K]: col-major, 全局索引 B[n*K + k] ( 注意内维是 K 不是 N!)
// shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
// - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
// - ldmatrix B: 用 x2 (非转置), 因为 B 已是 col-major (BLAS Normal), 无需 .trans
// - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
// - grid.z = S 个 swizzle 分区
// - grid.x = 每个分区内部需要发射多少个 128x128 的 N tiles
// - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N 区域, 改善
// L2 命中
// Block: (256, 1, 1), 8 warps
// source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
          const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
          const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
          const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
__global__ void __launch_bounds__(256)
    hgemm_mma_m16n8k16_mma2x4_warp4x4_stages_dsmem_tn_kernel(half *A, half *B,
                                                             half *C, int M,
                                                             int N, int K
    ) {

  // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
  const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
  const int by = blockIdx.y;
  constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
  constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
  constexpr int BK = MMA_K;                            // 16

  // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
  // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B col-major)
  // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
  // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简
  // PAD=0 版本。
  extern __shared__ half smem[];
  half *s_a = smem;
  half *s_b = smem + K_STAGE * BM * BK;     // A 和 B 连续存放
  constexpr int s_a_stage_offset = BM * BK; // 128*16
  constexpr int s_b_stage_offset =
      BN * BK; // 128*16    BN(128)×BK(16) = B col-major 的 smem 布局

  const int tid = threadIdx.y * blockDim.x + threadIdx.x;
  const int warp_id = tid / WARP_SIZE; // 0~7
  const int lane_id = tid % WARP_SIZE; // 0~31
  const int warp_m = warp_id % 2;      // 0,1 (M 方向 2 个 warp)
  const int warp_n = warp_id / 2;      // 0,1,2,3 (N 方向 4 个 warp)

  // 线程到 global memory 的映射 (用于加载 A 和 B)
  // TN 布局关键: A[m*K+k] 是 row-major, B[n*K+k] 是 col-major (内维是 K!)
  int load_smem_a_m = tid / 2;                // 0~127
  int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
  int load_smem_b_n = tid / 2; // 0~127 → B 的 N 方向 (col-major 的行)
  int load_smem_b_k =
      (tid % 2 == 0) ? 0 : 8; // 0, 8  → B 的 K 方向 (col-major 的列)
  int load_gmem_a_m = by * BM + load_smem_a_m;
  int load_gmem_b_n =
      bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏移
  if (load_gmem_a_m >= M || load_gmem_b_n >= N)
    return;

  // 累加器: 每个 warp 计算 WARP_TILE_M×WARP_TILE_N=16 个 MMA tile
  uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0

  // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
  uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
  uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);

  // 预加载前 (K_STAGE-1) 个 stage
  // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B 用 n*K+k(col-major)
#pragma unroll
  for (int k = 0; k < (K_STAGE - 1); ++k) {
    int load_gmem_a_k = k * BK + load_smem_a_k;
    int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
    int load_gmem_b_k = k * BK + load_smem_b_k;
    int load_gmem_b_addr =
        load_gmem_b_n * K + load_gmem_b_k; // B: [n][k] col-major

    uint32_t load_smem_a_ptr =
        (smem_a_base_ptr +
         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *
             sizeof(half));
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    uint32_t load_smem_b_ptr =
        (smem_b_base_ptr +
         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
             sizeof(half));
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);

    CP_ASYNC_COMMIT_GROUP();
  }

  const int NUM_K_TILES = div_ceil(K, BK);
  CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
  __syncthreads();

  // 主循环: K 维分块迭代
#pragma unroll
  for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
    // Stage 选择: 轮转方式 (round-robin)
    // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
    // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥。
    int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
    int smem_sel_next = k % K_STAGE;  // 下一轮加载的 stage

    // 异步加载下一批数据到 smem_sel_next
    // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B 的 gmem 地址用
    // n*K+k (col-major, 内维是 K)
    int load_gmem_a_k = k * BK + load_smem_a_k;
    int load_gmem_a_addr =
        load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
    int load_gmem_b_k = k * BK + load_smem_b_k;
```

```
    int load_gmem_b_addr =
        load_gmem_b_n * K + load_gmem_b_k; // B: col-major [n][k]  内维是 K!

    uint32_t load_smem_a_ptr =
        (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
                            load_smem_a_m * BK + load_smem_a_k) *
                              sizeof(half));
    CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

    uint32_t load_smem_b_ptr =
        (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
                            load_smem_b_n * BK + load_smem_b_k) *
                              sizeof(half));
    CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
    CP_ASYNC_COMMIT_GROUP();

    // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
    // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
    // B 用 x2 (非转置), 因为 B 已经是 col-major, 无需 .trans
    uint32_t RA[WARP_TILE_M][4];
    uint32_t RB[WARP_TILE_N][2];

    // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
#pragma unroll
    for (int i = 0; i < WARP_TILE_M; ++i) {
      int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
      int lane_smem_a_m =
          warp_smem_a_m + lane_id % 16;       // ldmatrix 用 16 个 lane
      int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
      uint32_t lane_smem_a_ptr =
          (smem_a_base_ptr +
           (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
            lane_smem_a_k) *
               sizeof(half));
      LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr
      );
    }

    // ldmatrix.x2: 加载 B 的 k16n8 片段 (col-major B, 非转置)
    // 为什么不用 .trans？因为 TN 布局下 B 已经是 col-major [N][K]
    // ldmatrix 默认期望 col-major 输入 → 直接匹配
#pragma unroll
    for (int j = 0; j < WARP_TILE_N; ++j) {
      int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
      int lane_smem_b_n =
          warp_smem_b_n + lane_id % 8;            // ldmatrix B 用 8 个 lane
      int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
      uint32_t lane_smem_b_ptr =
          (smem_b_base_ptr +
           (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
            lane_smem_b_k) *
               sizeof(half));
      LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
    }

    // MMA compute: 发射 WARP_TILE_M × WARP_TILE_N 条 MMA 指令
#pragma unroll
    for (int i = 0; i < WARP_TILE_M; ++i) {
#pragma unroll
      for (int j = 0; j < WARP_TILE_N; ++j) {
        HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
                  RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j
      ][1]);
      }
    }

    // 等待当前 stage 的异步加载完成, 然后 sync
    CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
    __syncthreads();
  }

  // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
  if ((K_STAGE - 2) > 0) {
    CP_ASYNC_WAIT_GROUP(0);
    __syncthreads();
  }

  {
#pragma unroll
    for (int k = 0; k < (K_STAGE - 1); k++) {
      uint32_t RA[WARP_TILE_M][4];
      uint32_t RB[WARP_TILE_N][2];
      int stage_sel = ((NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE);

#pragma unroll
      for (int i = 0; i < WARP_TILE_M; ++i) {
        int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
        int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
        int lane_smem_a_k = (lane_id / 16) * 8;
        uint32_t lane_smem_a_ptr =
            (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
                                lane_smem_a_m * BK + lane_smem_a_k) *
                                   sizeof(half));
        LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3],
                    lane_smem_a_ptr);
      }
#pragma unroll
      for (int j = 0; j < WARP_TILE_N; ++j) {
        int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
        int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
        int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
        uint32_t lane_smem_b_ptr =
            (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
                                lane_smem_b_n * BK + lane_smem_b_k) *
                                   sizeof(half));
        LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
      }
#pragma unroll
      for (int i = 0; i < WARP_TILE_M; ++i) {
#pragma unroll
        for (int j = 0; j < WARP_TILE_N; ++j) {
          HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i
      ][2],
                    RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j
      ][1]);
        }
      }
    }
  }

  // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
  {
    for (int i = 0; i < WARP_TILE_M; ++i) {
      uint32_t RC0[WARP_TILE_N][4];
      uint32_t RC1[WARP_TILE_N][4];
      // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
      // 对 m16n8k16 的 C fragment:
      // - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
      // - lane+1 持有同一行里的 {c2,c3}
      // - lane+2 持有同一行里的 {c4,c5}
      // - lane+3 持有同一行里的 {c6,c7}
      // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由 lane%4==0
      // 的那个线程做一次 128-bit store。
#pragma unroll
      for (int j = 0; j < WARP_TILE_N; ++j) {
        RC0[j][0] = RC[i][j][0];
        RC1[j][0] = RC[i][j][1];
        RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
        RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
        RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
        RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
        RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
        RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
      }
      // 每 4 个 lane 中只有 lane 0 做 128-bit store
      if (lane_id % 4 == 0) {
        int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i *
```

```
      MMA_M;
        int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id
       / 4;
#pragma unroll
        for (int j = 0; j < WARP_TILE_N; ++j) {
          int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j *
      MMA_N;
          int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
          int store_gmem_c_addr_0 =
              store_lane_gmem_c_m * N + store_lane_gmem_c_n;
          int store_gmem_c_addr_1 =
              (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
          // 128-bit store: 一次写入 8 个 half
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
              *reinterpret_cast<float4 *>(&RC0[j][0]);
          *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
              *reinterpret_cast<float4 *>(&RC1[j][0]);
        }
      }
    }
  }
}

// ==================================================
// Phase 5b-3: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (
      Hopper)
// ==================================================
// 面试要点 (WGMMA vs MMA 对比):
// - MMA: warp 级 (32 threads), 同步执行
// - WGMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-
      forget)
// - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA
      的 32
// 倍)
// - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
// - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMA, 通过
      barrier
// 同步
// - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict

// ---- WGMMA 辅助函数 ----
#define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "
      memory")
#define WGMMA_COMMIT_GROUP()
  asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
#define WGMMA_WAIT_GROUP(n)
  asm volatile("wgmma.wait_group.sync.aligned %0;\n" ::"n"(n) : "memory")

// Shared memory descriptor encode: 将 smem 地址编码为 WGMMA 可用的描述符
#define SMEM_DESC_ENCODE(x) ((((uint64_t)(x)) & 0x3FFFF) >> 0x4)

// make_smem_desc: 创建 WGMMA 的 shared memory 矩阵描述符
// 这里沿用原始 hgemm_wgmma 实现里的描述符字段约定:
// - base addr: 当前 shared memory tile 基址
// - leading offset: 16 bytes = 8 个 half * 2B, 对应 WGMMA 在 tile 内沿
      minor
// 方向前进一次的 byte 步长
// - stride offset: 1024 bytes = 64 * 8 * 2B, 对应跨到下一条 major stripe
      的 byte 步长
// - bit 62: 打开 128B swizzle
// 这些字段是当前 WGMMA/TMA 布局下的实现常量, 不要把它直接背成 BM/BK
// 的原始字节数公式。
__device__ inline uint64_t make_smem_desc(half *ptr) {
  uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
  uint64_t desc = 0x0000000000000000;
  desc |= SMEM_DESC_ENCODE(addr);
  desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;   // leading dim offset
      bytes
  desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32; // stride dim offset
      bytes
  desc |= 1llu << 62;                             // 128B swizzle
  return desc;
}

// ---- WGMMA PTX 指令宏 ----
// wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
// m64n128k16: M=64, N=128, K=16
// f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
// 32 个输出寄存器 = 64×128/2/4 = 1024 个 half / 32 = 32 个 uint32
// descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
// ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
#define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB,
      TransA,
                                   TransB)
 {
   uint64_t desc_a = make_smem_desc(&(sA)[0]);
   uint64_t desc_b = make_smem_desc(&(sB)[0]);
   asm volatile(
       "{\n"
       "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 "
       "{%0,   %1,   %2,   %3,   %4,   %5,   %6,   %7,   "
       " %8,   %9,   %10,  %11,  %12,  %13,  %14,  %15,  "
       " %16,  %17,  %18,  %19,  %20,  %21,  %22,  %23,  "
       " %24,  %25,  %26,  %27,  %28,  %29,  %30,  %31},"
       " %32,"
       " %33,"
       " %34, %35, %36, %37, %38;\n"
       "}\n"
       : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d
      )[0][3]),
         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d
      )[1][3]),
         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d
      )[2][3]),
         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d
      )[3][3]),
         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d
      )[4][3]),
         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d
      )[5][3]),
         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d
      )[6][3]),
         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d
      )[7][3])
       : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)),
         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA
      )),
         "n"(int32_t(TransB)));
 }

// ---- TMA Shared Memory Layout ----
// Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[
      BK×BN]
template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
  alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
  alignas(128)
      half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN] (即 col-
      major B^T)
};

// ---- WGMMA Kernel: Warp Specialization + TMA ----
// 面试重点 — Warp Specialization:
// WG0 (128 threads): Producer — 用 TMA 异步加载 A/B 到 shared memory
// WG1 (128 threads): Consumer — 用 WGMMA 做矩阵乘
// 同步: cuda::barrier (CTA 级别), Producer 发 full 信号, Consumer 发
      empty
// 信号 K_STAGE=3: 3 个 stage, Consumer 滞后 Producer 最多 2 步
// Grid:  ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block
      swizzle
// Block: (256, 1, 1), 2 warpgroups(Producer+Consumer)
// source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
template <const int WGMMA_M = 64, const int WGMMA_N = 128,
          const int WGMMA_K = 16, const int BM = 128, const int BN = 128,
          const int BK = 64, const int NUM_THREADS = 256, const int
      K_STAGE = 3,
          const bool BLOCK_SWIZZLE = false>
__global__ void __launch_bounds__(NUM_THREADS)
    hgemm_wgmma_m64n128k16_f16acc_stages_tma_ws_tn_kernel(
        int M, int N, int K, half *C,
        const CUtensorMap *__restrict__ tensorMapA,
```

```
      const CUtensorMap *__restrict__ tensorMapB) {

// 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
// 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
// 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
// 不展开宿主侧 create_tensor_map 细节。

const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
const int by = blockIdx.y;
constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
constexpr int B_WG_M = BM / num_consumers;             // 128

if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
  return;

extern __shared__ __align__(128) uint8_t smem[];
WgmmaSMem<BM, BN, BK, K_STAGE> &s =
    *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem);
half *s_a = s.A;
half *s_b = s.B;

// CTA barrier: 同步 Producer 和 Consumer
__shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
__shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];

const int num_blocks_k = K / BK;
const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
const int tid = threadIdx.x % 128;    // 0~127 within warpgroup

// 初始化 barriers (仅 thread 0 执行)
if (threadIdx.x == 0) {
  for (int i = 0; i < K_STAGE; ++i) {
    // 这里的 129 不是 "warp 数" 也不是 "线程块总线程数",
    // 而是这个 barrier 上每轮会 arrive 的参与者总数: 128 个 consumer 线程 + 1 个
    // producer 提交线程。
    init(&full[i], num_consumers * 128 + 1);  // 128 consumer + 1 producer
    init(&empty[i], num_consumers * 128 + 1); // same
  }
  cuda::device::experimental::fence_proxy_async_shared_cta();
}
__syncthreads();

// ========== Producer Warpgroup (WG0) ==========
if (wg_idx == 0) {
  if (tid == 0) { // 仅 producer 的 thread 0 执行 TMA 加载
    // TMA 是硬件 DMA 提交指令: 发起一次 descriptor+坐标提交后,
    // 后续搬运由硬件异步完成,
    // 不需要整个 producer warpgroup 里的 128 个线程都参与拷贝。
    int qidx = 0;
    for (int block_k_iter = 0; block_k_iter < num_blocks_k;
         ++block_k_iter, ++qidx) {
      if (qidx == K_STAGE)
        qidx = 0;

      // 等待 Consumer 释放此 stage (empty 信号)
      empty[qidx].wait(empty[qidx].arrive());

      // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
      cuda::device::experimental::
    cp_async_bulk_tensor_2d_global_to_shared(
          &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
          full[qidx]);

      // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
      cuda::device::experimental::
    cp_async_bulk_tensor_2d_global_to_shared(
          &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
          full[qidx]);

      // 通知 Consumer 此 stage 已准备好 (full 信号)
      cuda::device::barrier_arrive_tx(full[qidx], 1,
                                      (BK * BN + BK * BM) * sizeof(half
    ));
    }
  }
// ========== Consumer Warpgroup (WG1) ==========
else {
  // Consumer 初始时"准备就绪", arrive 到所有 empty barriers
  for (int i = 0; i < K_STAGE; ++i) {
    empty[i].arrive();
  }

  // 累加器: B_WG_M/WGMMA_M=2 行 × WGMMA_N/16=8 列 = 16 组寄存器
  uint32_t d[B_WG_M / WGMMA_M][WGMMA_N / 16][4] = {};

  int qidx = 0;
  for (int block_k_iter = 0; block_k_iter < num_blocks_k;
       ++block_k_iter, ++qidx) {
    if (qidx == K_STAGE)
      qidx = 0;

    // 等待 Producer 的 full 信号
    full[qidx].wait(full[qidx].arrive());

    // WGMMA 指令序列的常见记法是:
    // FENCE -> 发射一串 WGMMA -> COMMIT_GROUP -> WAIT_GROUP -> 下一轮再 FENCE
    // fence 是为后续 WGMMA 建立可见性/顺序关系; commit 表示当前这组 WGMMA
    // 已经发完;
    // wait 才是真正等待已 commit 的异步矩阵乘完成。
    // WGMMA fence: 确保 smem 写可见、accum 寄存器准备好
    WGMMA_FENCE();

    // M 维迭代: BM/WGMMA_M = 128/64 = 2
#pragma unroll
    for (int m_it = 0; m_it < B_WG_M / WGMMA_M; ++m_it) {
      half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMA_M;

      // K 维迭代: BK/WGMMA_K = 64/16 = 4
#pragma unroll
      for (int k_it = 0; k_it < BK / WGMMA_K; ++k_it) {
        WGMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMA_K,
                                   s_b + qidx * BK * BN + k_it *
    WGMMA_K,
                                   1, // ScaleD=1: accumulate (不清零)
                                   1, 1, 0, 0);
      }
    }

    WGMMA_COMMIT_GROUP();
    WGMMA_WAIT_GROUP(0); // 等待所有 WGMMA 完成

    // 释放此 stage (发 empty 信号给 Producer)
    empty[qidx].arrive();
  }

  // ===== Epilogue: 写回 C =====
  // WGMMA m64n128k16 的输出寄存器映射可这样记:
  // row = warp * 16 + lane / 4
  // col = g * 16 + 2 * (lane % 4)
  // d[m_it][g][0] -> (row,     col)
  // d[m_it][g][1] -> (row + 8, col)
  // d[m_it][g][2] -> (row,     col + 8)
  // d[m_it][g][3] -> (row + 8, col + 8)
  // 所以每个线程一次写回 4 个 half2, 刚好覆盖当前 16x16 子块里的四个象限位置。
  const int lane = tid % 32;
  const int warp = tid / 32;
  const int row = warp * 16 + lane / 4;
  half *block_C = C + by * BM * N + bx * BN;

#pragma unroll
  for (int m_it = 0; m_it < B_WG_M / WGMMA_M; ++m_it) {
    int yo = m_it * WGMMA_M;
```

```
#pragma unroll
      for (int g = 0; g < WGMMA_N / 16; ++g) {
        int col = g * 16 + 2 * (lane % 4);
        // 将 uint32 直接 reinterpret 为 half2 pair 写入 C
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col]) =
            d[m_it][g][0];
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col])
        =
            d[m_it][g][1];
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo) * N + col + 8])
        =
            d[m_it][g][2];
        *reinterpret_cast<uint32_t *>(&block_C[(row + yo + 8) * N + col +
        8]) =
            d[m_it][g][3];
      }
    }
  }
}

// ================================================
// Phase 6: RoPE — 旋转位置编码 (Rotary Position Embedding)
// ================================================
// 面试要点:
// - RoPE 数学公式: 对每对相邻维度做 2D 旋转
// [x1']   [cos()  -sin()] [x1]
// [x2'] = [sin()   cos()] [x2]
// -  _i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
// - token_pos = idx / N: token 在序列中的位置
// - token_idx = idx % N: token 内的维度对索引
// - 输入 [seq_len, hidden_size], 输出同形状

// Grid:  ((seq_len * N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/rope/rope.cu
__global__ void rope_f32_kernel(float *x, float *out, int seq_len, int N)
    {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  float x1 = x[idx * 2];
  float x2 = x[idx * 2 + 1];
  int token_pos = idx / N; // 序列位置
  int token_idx = idx % N; // 维度对索引

  // 频率计算:  _i = 1 / (10000^(2i/d))
  float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
  float sin_v = sinf(token_pos * exp_v);
  float cos_v = cosf(token_pos * exp_v);

  // 2D 旋转
  float out1 = x1 * cos_v - x2 * sin_v;
  float out2 = x1 * sin_v + x2 * cos_v;
  out[idx * 2] = out1;
  out[idx * 2 + 1] = out2;
}

// ================================================
// Phase 7: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
// ================================================
// 面试要点 (Bank Conflict 专题):
// - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
// - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
// - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
// - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
//
// 转置的三步演进:
// naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
// shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
// BCF:   smem 布局 [WARP_SIZE_S][WARP_SIZE_S*4+PAD], PAD=1 消除 bank
    conflict
// merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store

// ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写)----
// 每个线程处理 1 个元素, block(16,16)
// 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取
// 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入  (32 次内存事务)
// Grid:  ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
// Block: (16, 16, 1)
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void mat_transpose_f32_row2col2d_kernel(float *x, float *y,
                                                   const int row,
                                                   const int col) {
  const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
  const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
  if (global_y < row && global_x < col) {
    // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优
    先, 合并
    // 写入: y[global_y][global_x] = y[global_y * row +
    // global_x], 列优先, 非合并
    y[global_y * row + global_x] = x[global_x * col + global_y];
  }
}

// ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write)
    ----

// Grid:  ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
// Block: (16, 16, 1)
// 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
// source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
__global__ void
    mat_transpose_f32x4_shared_bcf_merge_write_row2col2d_kernel(
    float *x, float *y, const int row, const int col) {
  const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
  const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
  const int local_x = threadIdx.x;
  const int local_y = threadIdx.y;

  constexpr int WARP_SIZE_S = 16;
  constexpr int PAD = 1;
  // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
  __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];

  if (global_y * 4 < row && global_x < col) {
    // Step 1: 从 global memory 读取 4 行, 写入 shared
    // memory (行优先, 合并读取)
    float4 x_val;
    x_val.x = x[(global_y * 4) * col + global_x];
    x_val.y = x[(global_y * 4 + 1) * col + global_x];
    x_val.z = x[(global_y * 4 + 2) * col + global_x];
    x_val.w = x[(global_y * 4 + 3) * col + global_x];
    tile[local_y * 4][local_x] = x_val.x;
    tile[local_y * 4 + 1][local_x] = x_val.y;
    tile[local_y * 4 + 2][local_x] = x_val.z;
    tile[local_y * 4 + 3][local_x] = x_val.w;
    __syncthreads();

    // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
    float4 smem_val;
    smem_val.x = tile[local_x * 4][local_y];
    smem_val.y = tile[local_x * 4 + 1][local_y];
    smem_val.z = tile[local_x * 4 + 2][local_y];
    smem_val.w = tile[local_x * 4 + 3][local_y];

    const int gid_x = blockIdx.x * blockDim.x;
    const int gid_y = blockIdx.y * blockDim.y * 4;
    const int out_y = gid_y + local_x * 4;
    const int out_x = gid_x + local_y;

    // float4 merge write: 1 次 128-bit store; 这里也隐含 out_y 为 4 的倍
    数
    *reinterpret_cast<float4 *>(&y[(out_x * row + out_y) / 4]) =
        *reinterpret_cast<float4 *>(&smem_val);
  }
}

// ================================================
// Phase 8: 杂项 Ops — Dot Product / Block All Reduce / Histogram
// ================================================
// 面试要点:
// - Dot Product: elementwise mul + reduce, 演示两种 reduce 模式结合
```

```
// - Block All Reduce: 多 block 各自做 reduce，最后 atomicAdd 到全局结果
// - Histogram: atomicAdd 模式，演示共享内存冲突的处理

// ---- Dot Product: y = sum(a[i] * b[i]) ----
// 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
template <const int NUM_THREADS = 128>
// Grid:  ((N + 127) / 128, 1, 1)
// Block: (128, 1, 1)
// source: LeetCUDA/kernels/dot-product/dot_product.cu
__global__ void dot(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * NUM_THREADS + tid;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  __shared__ float reduce_smem[NUM_WARPS];

  float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  prod = warp_reduce_sum<WARP_SIZE>(prod);
  if (lane == 0)
    reduce_smem[warp] = prod;
  __syncthreads();

  prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0)
    prod = warp_reduce_sum<NUM_WARPS>(prod);
  if (tid == 0)
    atomicAdd(y, prod);
}

// Dot Product + float4
template <const int NUM_THREADS = 128 / 4>
// Grid:  ((N + 127) / 128, 1, 1)
// Block: (32, 1, 1), 128/4=32
// 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/dot-product/dot_product.cu
__global__ void dot_vec4(float *a, float *b, float *y, int N) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  __shared__ float reduce_smem[NUM_WARPS];

  float4 reg_a = FLOAT4(a[idx]);
  float4 reg_b = FLOAT4(b[idx]);
  float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
                            reg_a.z * reg_b.z + reg_a.w * reg_b.w)
                         : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  prod = warp_reduce_sum<WARP_SIZE>(prod);
  if (lane == 0)
    reduce_smem[warp] = prod;
  __syncthreads();

  prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0)
    prod = warp_reduce_sum<NUM_WARPS>(prod);
  if (tid == 0)
    atomicAdd(y, prod);
}

// ---- Block All Reduce Sum: y = sum(a[0..N-1]) ----
// 多 block 各自做 warp→smem→warp0 reduce，然后 atomicAdd 到全局 y
template <const int NUM_THREADS = 128>
// Grid:  ((N + 127) / 128, 1, 1)
// Block: (128, 1, 1)
// source: LeetCUDA/kernels/reduce/block_all_reduce.cu
__global__ void block_all_reduce_sum(float *a, float *y, int N) {
  int tid = threadIdx.x;
  int idx = blockIdx.x * NUM_THREADS + tid;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  __shared__ float reduce_smem[NUM_WARPS];

  float sum = (idx < N) ? a[idx] : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  sum = warp_reduce_sum<WARP_SIZE>(sum);
  if (lane == 0)
    reduce_smem[warp] = sum;
  __syncthreads();

  sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0)
    sum = warp_reduce_sum<NUM_WARPS>(sum);
  if (tid == 0)
    atomicAdd(y, sum);
}

// Block All Reduce Sum + float4
template <const int NUM_THREADS = 128 / 4>
// Grid:  ((N + 127) / 128, 1, 1)
// Block: (32, 1, 1), 128/4=32
// 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/reduce/block_all_reduce.cu
__global__ void block_all_reduce_sum_vec4(float *a, float *y, int N) {
  int tid = threadIdx.x;
  int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
  constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
  __shared__ float reduce_smem[NUM_WARPS];

  float4 reg_a = FLOAT4(a[idx]);
  float sum = (idx < N) ? (reg_a.x + reg_a.y + reg_a.z + reg_a.w) : 0.0f;
  int warp = tid / WARP_SIZE;
  int lane = tid % WARP_SIZE;

  sum = warp_reduce_sum<WARP_SIZE>(sum);
  if (lane == 0)
    reduce_smem[warp] = sum;
  __syncthreads();

  sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
  if (warp == 0)
    sum = warp_reduce_sum<NUM_WARPS>(sum);
  if (tid == 0)
    atomicAdd(y, sum);
}

// ---- Histogram: y[a[i]]++ ----
// 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (256, 1, 1)
// source: LeetCUDA/kernels/histogram/histogram.cu
__global__ void histogram(int *a, int *y, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N)
    atomicAdd(&(y[a[idx]]), 1);
}

// Histogram + int4 向量化
// Grid:  ((N + 255) / 256, 1, 1)
// Block: (64, 1, 1), 256/4=64
// 注意: 该版本默认输入地址满足 int4 对齐; 最适合 N 按 4 对齐的场景
// source: LeetCUDA/kernels/histogram/histogram.cu
__global__ void histogram_vec4(int *a, int *y, int N) {
  int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
  if (idx < N) {
    int4 reg_a = INT4(a[idx]);
    atomicAdd(&(y[reg_a.x]), 1);
    atomicAdd(&(y[reg_a.y]), 1);
    atomicAdd(&(y[reg_a.z]), 1);
    atomicAdd(&(y[reg_a.w]), 1);
  }
}

// ================================================
// Phase 9: FlashAttention-2 (Split-Q + MMA m16n8k16)
// ================================================
```

```cpp
// 面试要点 (FlashAttention 算法):
// 1. 核心问题: 标准 Attention 的 O(N^2) 中间矩阵 (S=QK^T) 必须写入 HBM,
// 但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
// 2. FA 三板斧:
// a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
// b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
// c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
// 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
// - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
// - 优点: 减少 warp 间通信和 shuffle
// 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):
// for each K,V tile:
// S_cur = Q @ K^T * scale
// m_new = max(m_old, row_max(S_cur))
// P_cur = exp(S_cur * scale - m_new)           // ← 写回 R_S 寄存器!
// l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
// O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
// O_final = O_new / l_final
// 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
// - R_S 经过 softmax 后, 存储的是 P = exp(S - m), 数据仍然是 half 精度
// - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
// 后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
// - 这是此实现的寄存器布局复用技巧, 不要背成 "所有 MMA A/C fragment
// 都天然同构" 的通用结论
//
// 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
// 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
// Grid:  ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
// Block: (128, 1, 1), kNumThreads=
    WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
// source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.
    cu

// ---- 寄存器填充辅助函数 ----
template <typename T, int M, const int N, const int K = 2>
__device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
#pragma unroll
      for (int k = 0; k < K; ++k)
        R[i][j][k] = val;
}

template <typename T, int M, const int N = 2>
__device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
#pragma unroll
  for (int i = 0; i < M; ++i)
#pragma unroll
    for (int j = 0; j < N; ++j)
      R[i][j] = val;
}

// ================================================
// FlashAttention-2 Split-Q Kernel (完整实现)
// ================================================
// Q,K,V,O: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
//
// Tile 设计 (以 kHeadDim=64 为例) :
// Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ = 16*4*1 = 64
// Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK = 8*1*8  = 64
// Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
//
// 执行流程:
// 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
// 2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
// 3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
// 4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
// 5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale -
    max)
// → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
// 6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S
    做 A
// → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
// 7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
// 8) 最终 rescale: O_final = (1/l_final) * O_final
// 9) Epilogue: warp shuffle + 128-bit collective store

template <
    const int kHeadDim,          // head dim: 32, 64, 128
    const int kMmaAtomM,         // 16 (MMA instruction M dimension)
    const int kMmaAtomN,         // 8  (MMA instruction N dimension)
    const int kMmaAtomK,         // 16 (MMA instruction K dimension)
    const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br
     =16*4=64
    const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc
     basis=8
    const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal
                                 // kMmaTileSeqLenQ
    const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim
     direction)
    const int kWarpTileSeqLenQ,  // warp tiles along Q's M, 1 → Br per
     warp=16
    const int kWarpTileSeqLenK,  // warp tiles along K's N, 8 → Bc_warp
     =8*8=64
    const int kWarpTileSeqLenP,  // warp tiles for P@V M dim, 1
    const int kWarpTileHeadDimV, // warp tiles for P@V N dim,
                                 // kHeadDim/(8*kMmaTileHeadDimV)
    const int kStage,            // pipeline stages for K: 1 or 2
    const int kPad>              // padding for bank conflict avoidance
__global__ void __launch_bounds__(WARP_SIZE *kMmaTileSeqLenQ *
    kMmaTileSeqLenK)
  flash_attn_mma_stages_split_q_kernel(half *Q, half *K, half *V, half
    *O,
                                       int QKV_seqlen, int QKV_head) {

  // Tile dimensions
  constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ; //
     64
  constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK; //
     64
  constexpr int kNumThreads =
      WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
  const int Tc = (QKV_seqlen + Bc - 1) / Bc;
  // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处
     理。
  // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理
     了尾 tile。
  const float scale = 1.0f / sqrtf((float)kHeadDim);

  // Block indexing
  const int QKV_batch_id = blockIdx.y / QKV_head;
  const int QKV_head_id = blockIdx.y % QKV_head;
  const int Q_tile_id = blockIdx.x;
  const int tid = threadIdx.x;
  const int warp_id = tid / WARP_SIZE;
  const int lane_id = tid % WARP_SIZE;
  const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
  const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)

  // Global memory base offsets for this (batch, head)
  // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
  const int Q_gmem_offset =
      (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
      kHeadDim;
  const int K_gmem_offset = Q_gmem_offset;
  const int V_gmem_offset = Q_gmem_offset;
  const int O_gmem_offset = Q_gmem_offset;

  // Thread-to-smem mapping for cooperative load
  int load_smem_Q_Br = tid / (kNumThreads / Br);
  int load_smem_Q_d =
      (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
  int load_smem_K_Bc = tid / (kNumThreads / Bc);
  int load_smem_K_d =
      (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
  int load_smem_V_Bc = tid / (kNumThreads / Bc);
  int load_smem_V_d =
      (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
```

```

  int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
  if (load_gmem_Q_Br >= QKV_seqlen)
    return;

  // ---- Shared memory layout ----
  extern __shared__ half smem[];
  constexpr int Q_tile_size = Br * (kHeadDim + kPad);  // Q tile: [Br, d+kPad]
  constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
  half *Q_tile_smem = smem;
  half *K_tile_smem = Q_tile_smem + Q_tile_size;
  half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
  // 原始 kernel 还留了一个优化点：若 kStage=1，K 和 V 在时序上并不重叠，
  // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。

  uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
  uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
  uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);

  // ---- Online Softmax persistent state ----
  // lane_block_row_max_old[i][r]: running max for row r of warp tile i
  // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile
  // i
  float lane_block_row_max_old[kWarpTileSeqLenQ][2];
  float lane_block_row_sum_old[kWarpTileSeqLenQ][2];
  fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
  fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

  // ---- Register allocation ----
  uint32_t R_Q[kWarpTileSeqLenQ][4];                    // Q regs
  uint32_t R_K[kWarpTileSeqLenK][2];                    // K regs
  uint32_t R_V[kWarpTileHeadDimV][2];                   // V regs
  // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
  // 对单个 m16n8k16 tile 而言：
  // - reg[0] 持有该 tile 前 8 行里的两个 half 值
  // - reg[1] 持有该 tile 后 8 行里的两个 half 值
  // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
  uint32_t R_S[kWarpTileSeqLenQ][kWarpTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
  uint32_t R_O[kWarpTileSeqLenP][kWarpTileHeadDimV][2]; // O for current tile
  uint32_t R_D[kWarpTileSeqLenP][kWarpTileHeadDimV]
              [2]; // O accumulator (final output)

  fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
  fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_D, 0);

  // ================================================
  // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
  // ================================================
  {
    int load_gmem_Q_addr =
        Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
    uint32_t load_smem_Q_ptr =
        smem_Q_base_ptr +
        (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(
      half);
#pragma unroll
    for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
      CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
    }
    CP_ASYNC_COMMIT_GROUP();
  }

  // ================================================
  // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
  // 注意：Q 由 blockIdx.x 固定到当前 Q tile；而 K/V 的 seqlen 遍历始终从 tile 0
  // 开始，
  // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
  // ================================================
  if constexpr (kStage > 1) {
#pragma unroll
    for (int stage = 0; stage < (kStage - 1); ++stage) {
      int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
      int load_gmem_K_addr =
          K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
      uint32_t load_smem_K_ptr =
          smem_K_base_ptr +
          (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
           load_smem_K_d) *
              sizeof(half);
#pragma unroll
      for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
        CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
      }
      CP_ASYNC_COMMIT_GROUP();
    }
    CP_ASYNC_WAIT_GROUP(kStage - 2);
    __syncthreads();
  }

  // ================================================
  // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
  // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
  // ================================================
#pragma unroll 1
  for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
    int smem_sel = tile_K_seqlen % kStage;
    int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;

    // ---- 3a: 异步加载 K/V tile (多 stage pipeline)----
    if constexpr (kStage > 1) {
      // 只有 kStage>1 才能真正做 K 的 pipeline：
      // smem_sel 负责“当前正在计算”的 K tile，smem_sel_next 负责“下一轮预取”的 K
      // tile。
      // 若 kStage=1，这两个槽位永远都等于 0，当前 K 还没算完就无法安全覆盖同一块
      // smem。
      // Load current V tile (no pipeline for V — one stage is enough)
      {
        int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
        int load_gmem_V_addr =
            V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
        uint32_t load_smem_V_ptr =
            smem_V_base_ptr +
            (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof
        (half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i],
        16);
        }
        CP_ASYNC_COMMIT_GROUP();
      }

      // Prefetch next K tile (pipelined)
      if ((tile_K_seqlen + 1) < Tc) {
        int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
        int load_gmem_K_addr =
            K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
        uint32_t load_smem_K_ptr =
            smem_K_base_ptr +
            (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim +
        kPad) +
             load_smem_K_d) *
                sizeof(half);
#pragma unroll
        for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
          CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i],
        16);
        }
```

```
      CP_ASYNC_COMMIT_GROUP();
    }
  }

  // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
  fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
#pragma unroll
  for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
    // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
#pragma unroll
    for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
      int warp_smem_Q_Br =
          warp_QP * (kMmaAtomM * kWarpTileSeqLenQ) + i * kMmaAtomM;
      int lane_smem_Q_Br =
          warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
      int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
      uint32_t lane_smem_Q_ptr =
          smem_Q_base_ptr +
          (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
      LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
                  lane_smem_Q_ptr);
    }

    // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
    // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
#pragma unroll
    for (int j = 0; j < kWarpTileSeqLenK; ++j) {
      int warp_smem_K_Bc =
          warp_KV * (kMmaAtomN * kWarpTileSeqLenK) + j * kMmaAtomN;
      int lane_smem_K_Bc =
          warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
      int lane_smem_K_d =
          tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
      uint32_t lane_smem_K_ptr =
          smem_K_base_ptr +
          (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
           lane_smem_K_d) *
              sizeof(half);
      LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
    }

    // MMA: S[tile] += Q[tile] @ K^T[tile]
#pragma unroll
    for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
#pragma unroll
      for (int j = 0; j < kWarpTileSeqLenK; ++j) {
        HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
                  R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
                  R_S[i][j][1]);
      }
    }
  } // end loop over d
  __syncthreads();

  // ================================================
  // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to
  // R_S
  // ================================================
  // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
  // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0, c1}
  // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
  // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
  // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
  // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
  // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
  // Each (i, j) pair = one 16x8 MMA tile; there are kWarpTileSeqLenQ x
  // kWarpTileSeqLenK tiles.

  float lane_row_max_new[kWarpTileSeqLenQ][2];
  float lane_row_sum_new[kWarpTileSeqLenQ][2];
  fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
  fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);

  // ---- Pass 1: Thread-level reduce max across all columns (kWarpTileSeqLenK
  // tiles) ----
#pragma unroll
  for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
#pragma unroll
    for (int j = 0; j < kWarpTileSeqLenK; ++j) {
      // Extract half values from R_S registers (C matrix fragment layout)
      float2 t_reg_S_0 =
          __half22float2(HALF2(R_S[i][j][0])); // rows 0~7:  {c0, c1}
      float2 t_reg_S_1 =
          __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
      // S = (Q@K^T) * scale
      float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
      float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
      lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
      lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
    }
    // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
    // {0,4,8,...,28} hold valid data)
    lane_row_max_new[i][0] =
        warp_reduce_max<float, 4>(lane_row_max_new[i][0]);
    lane_row_max_new[i][1] =
        warp_reduce_max<float, 4>(lane_row_max_new[i][1]);
  }

  // ---- Pass 2: Compute P = exp(S*scale - m_new), store back to R_S ----
  // 面试关键点: 这里将 P 写回 R_S 寄存器!
  // 为什么可以？当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
  // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
#pragma unroll
  for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
    // m_new = max(m_old, m_cur)
    float block_row_max_new_0 =
        max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
    float block_row_max_new_1 =
        max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);

#pragma unroll
    for (int j = 0; j < kWarpTileSeqLenK; ++j) {
      float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
      float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));

      // P = exp(S * scale - m_new), 用 fma 保证精度
      t_reg_S_0.x =
          __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
      t_reg_S_0.y =
          __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
      t_reg_S_1.x =
          __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
      t_reg_S_1.y =
          __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));

      // Accumulate row sums
      lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
      lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);

      // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
      HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
      HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
    }
```

```
    // Warp-level reduce sum (warp_size = 4, same as max)
    lane_row_sum_new[i][0] =
        warp_reduce_sum<float, 4>(lane_row_sum_new[i][0]);
    lane_row_sum_new[i][1] =
        warp_reduce_sum<float, 4>(lane_row_sum_new[i][1]);
  }
  __syncthreads();

  // ================================================
  // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
  // ================================================
  // Wait for V to be ready before computing P@V
  if constexpr (kStage > 1) {
    if ((tile_K_seqlen + 1) < Tc) {
      CP_ASYNC_WAIT_GROUP(1);
    } else {
      CP_ASYNC_WAIT_GROUP(0);
    }
  } else {
    CP_ASYNC_WAIT_GROUP(0);
  }
  __syncthreads();

  fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_O,
   0);

  // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
  // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim 64 configurations
#pragma unroll
  for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
    // ldmatrix.x2.trans: load V[Bc,d] with transposition
    // V is row-major [Bc,d], but NN matmul needs B matrix in col-major
    → use
    // transposed ldmatrix
#pragma unroll
    for (int j = 0; j < kWarpTileHeadDimV; ++j) {
      int warp_smem_V_d =
          warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
      int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
      int lane_smem_V_d = warp_smem_V_d;
      uint32_t lane_smem_V_ptr =
          smem_V_base_ptr +
          (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof
    (half);
      LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
    }

    // P matrix layout in R_S[i][j][2]:
    // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
    // |   64x64   |      warp_KV 0       |
    // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
    // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
    // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
    // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
    // tile_V_Bc selects which 16-column slice of P to use:
    // tile_V_Bc=0 → cols  0:16 → MMA indices 0,1  → w=0
    // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3  → w=2
    // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5  → w=4
    // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7  → w=6
    // 对应的 MMA A fragment 布局可以这样记:
    // - rows 0~7:  lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一
    行, 依次类推
    // - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一
    行, 依次类推
    // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以
    直接对接,
    // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对
    所有 MMA
    // fragment 都无条件成立的通用结论。
    int w = tile_V_Bc * 2;
#pragma unroll
    for (int i = 0; i < kWarpTileSeqLenP; ++i) {
#pragma unroll
      for (int j = 0; j < kWarpTileHeadDimV; ++j) {
        HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
                  R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][
    w + 1][1],  // A fragment = P
                  R_V[j][0], R_V[j][1], // B fragment = V
                  R_O[i][j][0], R_O[i][j][1]); // C fragment output
      }
    }
  } // end for tile_V_Bc
  __syncthreads();

  // ================================================
  // 3e: Online rescaling — O_new = exp(m_old - m_new) * O_old + P@V
  // ================================================
  // 公式来源: FA2 paper Eq.(7-8)
  // 注意: FA2 论文中的公式有误 (取倒数而非 exp), 实际实现使用 exp(m_old -
  // m_new)
#pragma unroll
  for (int i = 0; i < kWarpTileSeqLenP; ++i) {
    float block_row_max_new_0 = lane_row_max_new[i][0];
    float block_row_max_new_1 = lane_row_max_new[i][1];
    float block_row_sum_new_0 = lane_row_sum_new[i][0];
    float block_row_sum_new_1 = lane_row_sum_new[i][1];

    float block_row_max_old_0 = lane_block_row_max_old[i][0];
    float block_row_max_old_1 = lane_block_row_max_old[i][1];

    block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0
    );
    block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1
    );

    // Handle first iteration: m_old = -inf, need to use m_new directly
    block_row_max_old_0 =
        (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0
    );
    block_row_max_old_1 =
        (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1
    );

    float rescale_o_factor_0 =
        __expf(block_row_max_old_0 - block_row_max_new_0);
    float rescale_o_factor_1 =
        __expf(block_row_max_old_1 - block_row_max_new_1);

    // Rescale O_old + Add P@V in one fused step
#pragma unroll
    for (int j = 0; j < kWarpTileHeadDimV; ++j) {
      // R_O / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
      // reg[0] -> rows 0~7 的 {c0,c1}
      // reg[1] -> rows 8~15 的 {c2,c3}
      float2 t_reg_O_0 = __half22float2(HALF2(R_O[i][j][0]));
      float2 t_reg_O_1 = __half22float2(HALF2(R_O[i][j][1]));
      float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
      float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));

      // O_new = exp(m_old - m_new) * O_old + P@V  (fused multiply-add)
      t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x,
    t_reg_O_0.x);
      t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y,
    t_reg_O_0.y);
      t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x,
    t_reg_O_1.x);
      t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y,
    t_reg_O_1.y);

      HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
      HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
    }

    // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
    float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
    float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
    lane_block_row_sum_old[i][0] = __fmaf_rn(
        rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
    lane_block_row_sum_old[i][1] = __fmaf_rn(
        rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
```

```
      // Update m
      lane_block_row_max_old[i][0] = block_row_max_new_0;
      lane_block_row_max_old[i][1] = block_row_max_new_1;
    }

    // Wait for next K tile to be ready in smem before next iteration
    if constexpr (kStage > 1) {
      if ((tile_K_seqlen + 1) < Tc) {
        CP_ASYNC_WAIT_GROUP(0);
      }
      __syncthreads();
    }
  } // end outer loop over K seqlen
  __syncthreads();

  // ================================================
  // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
  // ================================================
#pragma unroll
  for (int i = 0; i < kWarpTileSeqLenP; ++i) {
    float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
    float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
#pragma unroll
    for (int j = 0; j < kWarpTileHeadDimV; ++j) {
      float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
      float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
      t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
      t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
      t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
      t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
      HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
      HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
    }
  }

  // ================================================
  // Step 5: Epilogue — Collective store via warp shuffle + 128-bit
     store
  // ================================================
  // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
  // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
#pragma unroll
  for (int i = 0; i < kWarpTileSeqLenP; ++i) {
#pragma unroll
    for (int j = 0; j < kWarpTileHeadDimV; ++j) {
      uint32_t R_Z[2][4];
      R_Z[0][0] = R_D[i][j][0];
      R_Z[1][0] = R_D[i][j][1];
      R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
      R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
      R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
      R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
      R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
      R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

      // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
      if (lane_id % 4 == 0) {
        int store_warp_regs_O_Br =
            warp_QP * (kMmaAtomM * kWarpTileSeqLenP) + i * kMmaAtomM;
        int store_lane_gmem_O_Br =
            Q_tile_id * Br + store_warp_regs_O_Br + lane_id / 4;
        int store_warp_regs_O_d =
            warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
        int store_lane_gmem_O_d = store_warp_regs_O_d;
        int store_gmem_O_addr_0 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 0) * kHeadDim +
                                  store_lane_gmem_O_d;
        int store_gmem_O_addr_1 = O_gmem_offset +
                                  (store_lane_gmem_O_Br + 8) * kHeadDim +
                                  store_lane_gmem_O_d;
        // LDST128BITS = reinterpret_cast<float4*>
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_0]) =
            *reinterpret_cast<float4 *>(&R_Z[0][0]);
        *reinterpret_cast<float4 *>(&O[store_gmem_O_addr_1]) =
            *reinterpret_cast<float4 *>(&R_Z[1][0]);
      }
    }
  }
}

// ================================================
// End of notes-v2.cu
// ================================================
// 本文件覆盖了面试中最高频的 CUDA kernel 考点 (37 个 kernel):
//   基础原语: warp_reduce (O(logN) butterfly), block_reduce (两级:
// warp→smem→warp0)   优化手段: coalescing, tiling, thread tile, vectorize
     ,
// pipeline, tensor core   Softmax 递进: naive → safe(2-pass) → online(1-
     pass,
// FA 基础)   GEMM 五层金字塔: tiling → thread tile → vectorize → MMA(TN布
     局)
// → WGMMA(warp spec)   FlashAttention: split-Q + online softmax + R_S
// 寄存器复用 P@V   Bank Conflict: 原理 + PAD 解决方案 + 四步演进   Memory
// Hierarchy: HBM → L2 → L1/SMEM → Register   BLAS 布局约定:
// N=col-major(Normal), T=row-major(Transposed), TN=A行B列
// ================================================
// 本文件覆盖了面试中最高频的 CUDA kernel 考点:
//   基础原语: warp_reduce (O(logN) butterfly), block_reduce (两级:
// warp→smem→warp0)   优化手段: coalescing, tiling, thread tile, vectorize
     ,
// pipeline, tensor core   Softmax 递进: naive → safe(2-pass) → online(1-
     pass,
// FA 基础)   GEMM 五层金字塔: tiling → thread tile → vectorize → MMA →
     WGMMA
//   FlashAttention: tiling + online softmax + recomputation 三板斧
//   Bank Conflict: 原理 + PAD 解决方案
//   Memory Hierarchy: HBM → L2 → L1/SMEM → Register
// ================================================
```